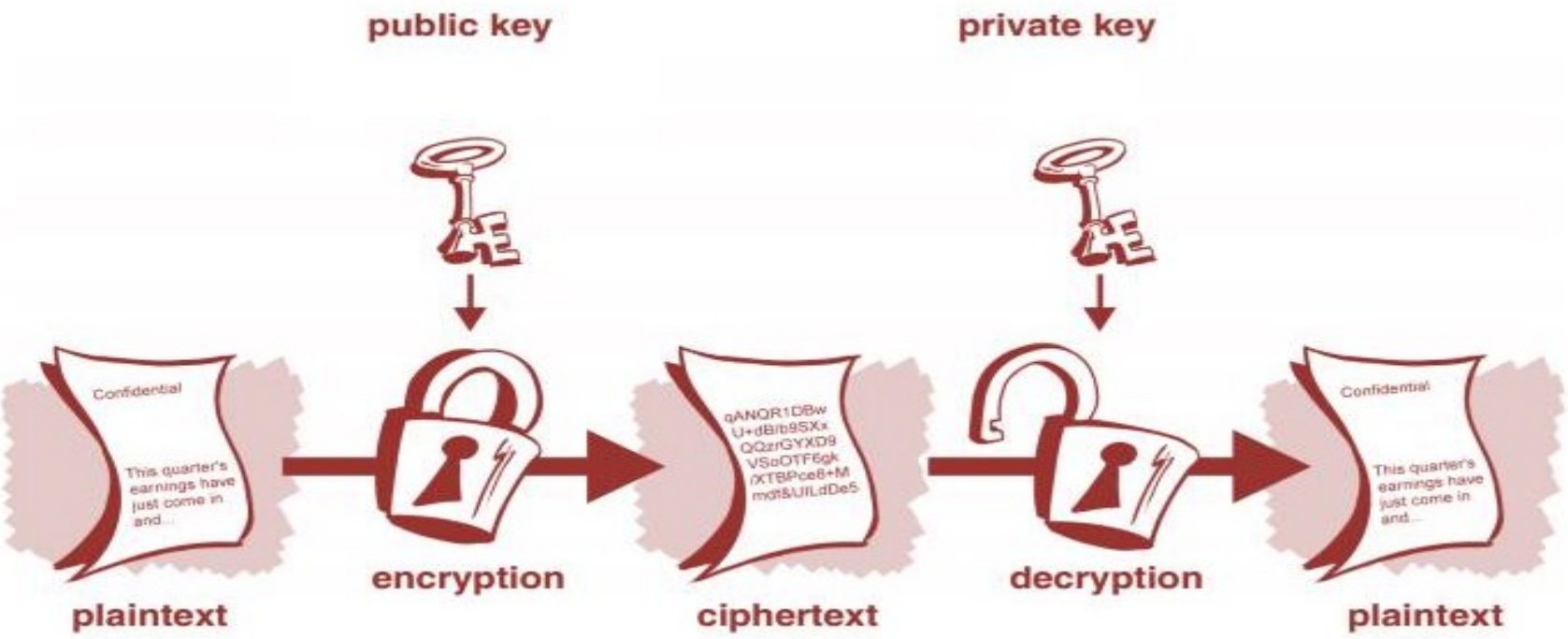
Хеш-алгоритми в криптографията

Доц. М. Иванова

https://medium.com/@shivamyadav91190/a-complete-guide-to-cryptographic-hashing-and-its-applications-securing-the-digital-realm-b5fb02bc259a

Асиметрична криптография



- Изисква по-голяма процесорна мощност

Защо хеширане?

- Необходимо е съобщението не само да достигне до получателя си, но и да достигне във формата, в който е изпратено.
- Как да се постигне това?
  - Чрез многократното му изпращане и верифициране, че не е променяно?
- Този подход отнема време и не е практичен (ако изпратеният файл е голям, тогава задачата се усложнява).
- Затова се прилага методът на хеширане.

Защо хеширане?

- **Хеширането** се извършва чрез алгоритъм, който от файл изчислява стринг с фиксиран размер в битове.
- Чрез хеширането файловите данни се трансформират в **много кратка стойност с фиксирана дължина.**
- Добрият хеш-алгоритъм трябва да се характеризира със свойство, наречено **лавинен ефект**, според който изходният хеш се променя значително, когато дори 1 бит входни данни е променен.
- Хешът е **шестнайсетичен стринг**.
- Хеширането е **еднопосочен процес** и никога от хеша не може да се достигне до явното съобщение.
- Хеш-алгоритъмът трябва да бъде достатъчно сложен, за да не произведе един и същи хеш-код за различни входни съобщения. Ако това се получи, тогава е налице **колизия** (т.е. алгоритъмът не е достатъчно добър).

Хеш-функци

- **Хеш-функцията** е математическа функция, която преобразува входната стойност в компресирана шестнадесетична стойност, наречена **хеш** или **хеш стойност.**
- Входните данни са с произволна дължина, а изходната хеш-функция винаги е с **фиксиран размер** (в зависимост от хеш-алгоритъма – от 160 до 512 бита.

Видове хеш-функции

Най-голямо приложение намират следните хеш-функции:

- MD4 (Message Digest 4)
- MD5 (Message Digest 5)
- SHA-1 (Secure Hash Algorithm)
- SHA-256
- SHA-384
- RIPEMD-128 (RACE (Research into Advanced Communications in Europe) Integrity Primitives Evaluation (RIPE Message Digest))
- RIPEMD-160

Видове хеш-функции

Хеш-алгоритъм	Година на създаване	Създател
MD2	1989	Ronald Rivest
MD4	1990	
MD5	1992	
MD6	2008	
RIPEMD	1992	The RIPE Consortium
RIPEMD-128 RIPEMD-256 RIPEMD-160 RIPEMD-320	1996	Hans Dobbertin Antoon Bosselaers Bart Preneel
SHA-0	1993	National Security Agency of the United States Department of Defense
SHA-1	1995	
SHA-256 SHA-384 SHA-512	2002	
SHA-224	2004	

Видове хеш-функции

Хеш-алгоритъм	Изходен размер, бит	Размер на блока, бит
MD2	128	128
MD4	128	512
MD5	128	512
RIPEMD	128	512
RIPEMD-128, -256	128/256	512
RIPEMD-160	160	512
RIPEMD-320	320	512
SHA-0	160	512
SHA-1	160	512
SHA-224, -256	224/256	512
SHA-384, -512, -512/224, -512/256	384/512/224/256	1024

Хеш-функции

- Хеш-функциите се комбинират с асиметрични криптографски алгоритми с цел проверка целостта на предаваните данни и потвърждаване на достоверността им.

Хеш-функция

За да се изпълни целта, хеш-функциите трябва да отговарят на изискванията:

- да могат да се прилагат за обработка на блокове с данни, **независимо от размера на блока**;
- да осигурят **хеш-код с фиксирана дължина**;
- да могат **лесно да се изпълняват**, незавимо дали реализацията е софтуерна или апаратна;
- трябва да са **еднопосочни функции,** т.е. за която и да е стойност на хеш-кода $h(X_i)$ на блока X_i да е невъзможно изчисляването на $h^{-1}(X_i)$;
- да се изпълнява свойството ***устойчивост на колизии,*** т.е. да е практически невъзможно изчисляването на $X_i \neq X_j$, за която $h(X_i)= h(X_j)$;
- да се изпълнява свойството ***силна устойчивост на колизии,*** т.е. да е практически невъзможно в резултат на хеширането на която и да е двойка X_i и X_j , за която $X_i \neq X_j$ да се получи $h(X_i)= h(X_j)$.

Хеш-функция

Три основни категории хеш-функции – в зависимост от реализацията:

- Създадени са въз основата на сложен математически апарат и изискват сложни изчисления (рядко се използват);
- Проектирани са по аналогия на симетричните блокови алгоритми и се реализират въз основа на блокови алгоритми (често се прилагат);
- Създадени са специално като еднопосочни функции в криптографията (позволяват ефективна програмна реализация).

Хеш-функция

- Всички хеш-функции се създават въз основа на принципи, които са общи и са предпоставка за успешната им реализация на практика:
  - Входната информация (съобщение, файл) се разделя на n-битови блокове.
  - Блоковете се обработват последователно, като се получава n-битова стойност на хеш-кода.
  - Най-простата функция за хеширане е последователното сумиране на блоковете по модул две:

$C_i=b_{i1} \oplus b_{i2} \oplus \ldots \oplus b_{im}$, C_i е i-тият бит от хеш-кода, $1<=i<=n$

m-брой на входните блокове, всеки с размер n-бита

b_{ij}, i-тият бит в j-тия блок

Пример

Задание: Нека входното съобщение е P и за обработката му се използва блоков алгоритъм, а K е секретен криптографски ключ

- **Предварителна дейност:** Съобщението P се разделя на блокове с определен размер t: P=X(1),X(2),…X(n). Къс блок се допълва до стандартен. Броят на блоковете е цяло число.
- **Хеширане:**

$E_K(X(1))=Y(1)$

$E_K(X(2)\oplus Y(1))=Y(2)$

…..

$E_K(X(n)\oplus Y(n-1))=Y(n)$

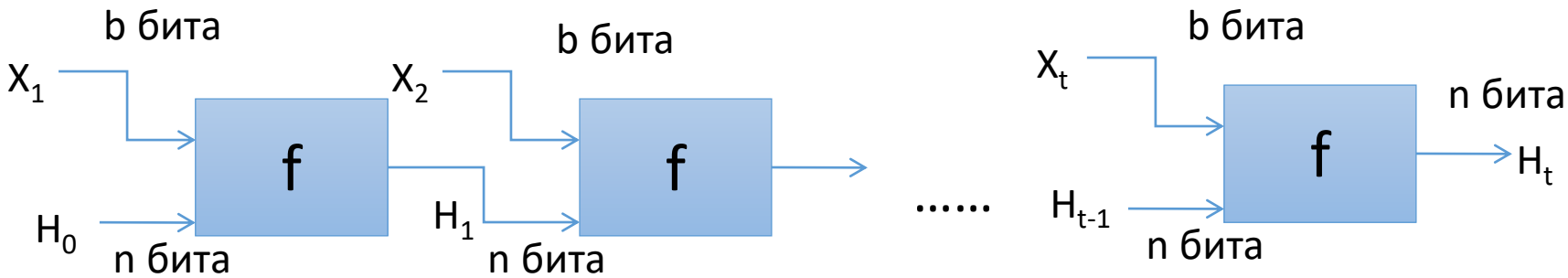
$E_K(X(1)\oplus X(2)\oplus \ldots X(n)\oplus Y(n))=h(P)$,

където $\oplus$ е сума по модул две (но може да бъде и сложна комплексна функция за криптиране). Хеш-кодът е с размер t.

Възможно е обработката на P да се извърши последователно с два секретни ключа K_1 и K_2 и тогава h(P) ще бъде конкантенацията (слепване) на $h_1(P)$ и $h_2(P)$ и полученият хеш-код ще е с размер 2t.

Обобщена структура на криптографска хеш-функция

Съобщението P е разделено на блокове $X_1, X_2, \ldots X_t$.



- H_0 – начална n-битова променлива;
- b бита е размерът на блоковете X;
- f – функция за компресиране;
- $H_i=f(H_{i-1},X_i)$, $1 \le i \le t$; $h(P)=H_t$

Онлайн изчисляване на хеш-функции: fileformat.info/tool/hash.htm

Алгоритъм MD5 за хеширане на входно съобщение P

1. Съобщението P се разделя на **512**-битови блокове, т.е. P=P(1)P(2)…P(n);
2. Всеки блок се разделя на **32-битови** 16 на брой подблока;
3. Ако се получи къс блок, той се допълва до 512 бита;
4. Извършва се инициализация на следните четири променливи:

A=01234567

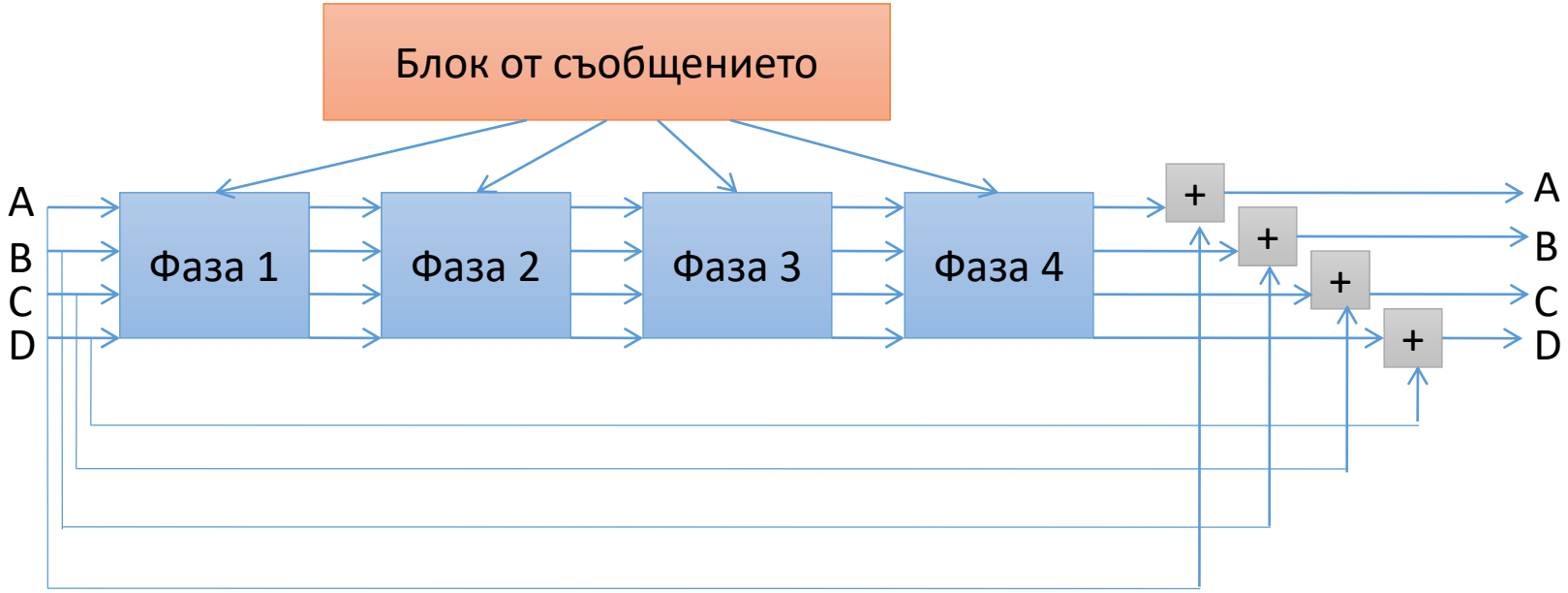
B=89abcdef

C=fedcba98

D=76543210

Алгоритъм MD5 за хеширане на входно съобщение P

- 5. Реализиране на алгоритъма като цикъл, състоящ се от четири еднакви фази.
- Във всяка фаза се извършват различни операции (общият им брой е 16 за всяка фаза или 64 за четирите фази).
- Цикълът се повтаря докато се обработят всички блокове на P.

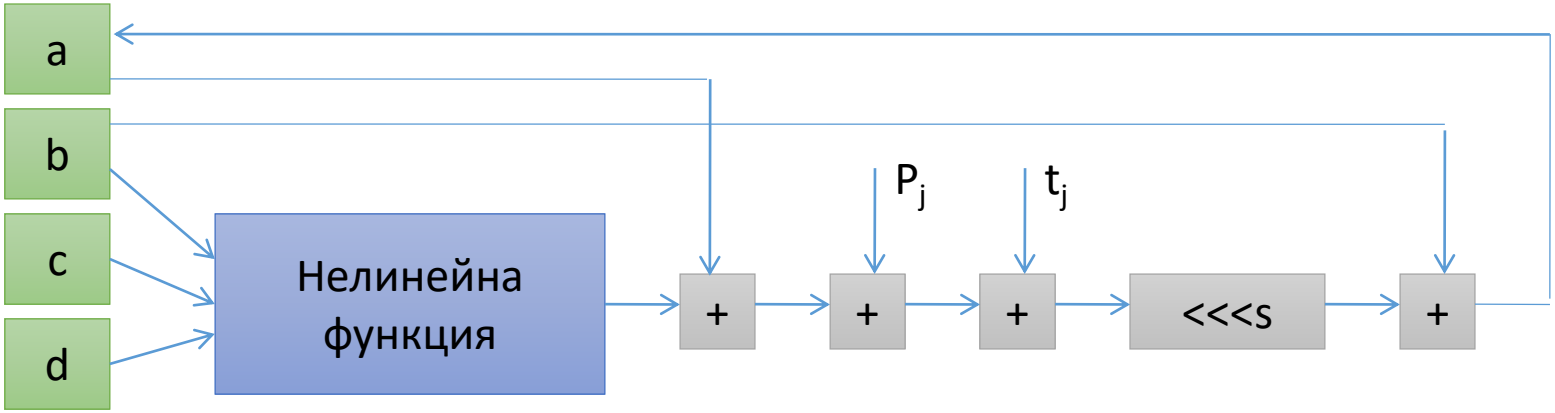


Алгоритъм MD5 за хеширане на входно съобщение P

6. Във **всяка фаза** се извършват операции. Цикълът се изпълнява докато се обработят всички блокове на P.

- Четирите променливи A,B,C,D се копират в помощни променливи a,b,c,d.
- Всяка операция представлява нелинейна функция на три от дефинираните четири променливи.
- Полученият резултат се сумира с четвъртата променлива, с подблок на текста P_j и с константа t_j.
- Извършва се циклично преместване наляво <<<s с s на брой бита и
- сумиране с една от променливите.
- Полученият резултат се използва за заместване на една от тези променливи.

<u>Схемата на **една операция** има вида</u>:



Алгоритъм MD5 за хеширане на входно съобщение P

- Във всяка от четирите фази се използва по една от следните нелинейни функции:

$$F(b,c,d) = (b \wedge c) \vee ((\neg b)d);$$

$$G(b,c,d) = (b \wedge d) \vee (c \vee (\neg d));$$

$$H(b,c,d) = b \oplus c \oplus d;$$

$$I(b,c,d) = c \oplus (b \vee (\neg d))$$

$\oplus$ XOR

където $\wedge$ - AND

$\vee$ - OR

$\neg$ - NOT

Алгоритъм MD5 за хеширане на входно съобщение P

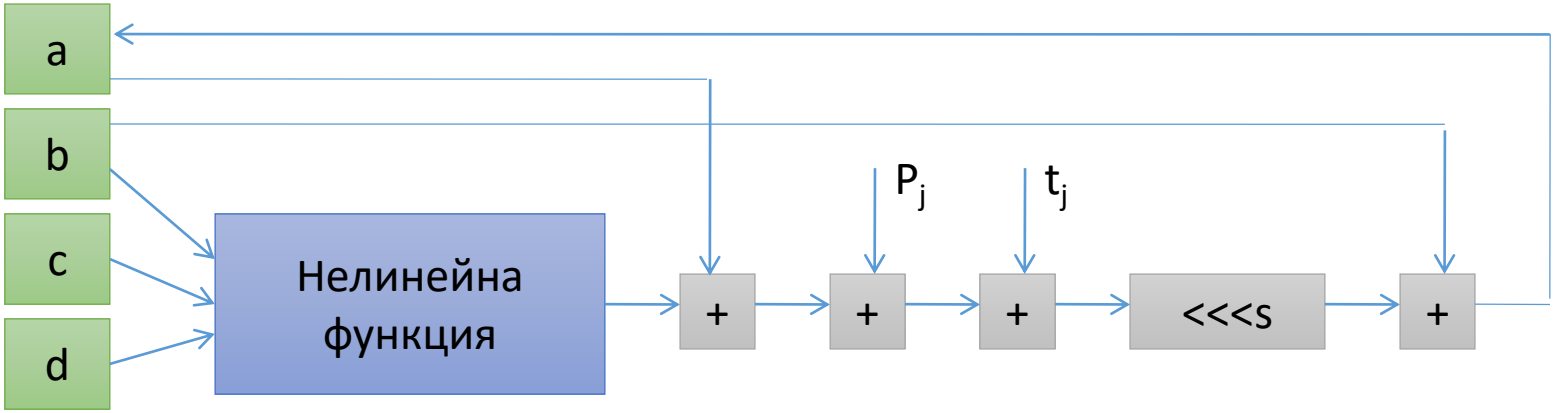
- Изпълняват се следните четири операции:

$$FF(a,b,c,d,p_j t_j) \rightarrow a = ((F(b,c,d) + a + p_j + t_j) <<< s) + b;$$

$$GG(a,b,c,d,p_j t_j) \rightarrow a = ((G(b,c,d) + a + p_j + t_j) <<< s) + b;$$

$$HH(a,b,c,d,p_j t_j) \rightarrow a = ((H(b,c,d) + a + p_j + t_j) <<< s) + b;$$

$$II(a,b,c,d,p_j t_j) \rightarrow a = ((I(b,c,d) + a + p_j + t_j) <<< s) + b;$$



Алгоритъм MD5 за хеширане на входно съобщение P

- Накрая a,b,c,d се сумират съответно с A,B,C,D и се преминава към обработка на следващия входен блок с данни.
- Същността на MD5 се състои във функцията за компресия на данните, която включва четирите фази и четирите логически функции F,G,H,I. Част от стойностите t_j са:

t	стойност
t[1]	D76AA478
t[2]	E8C7B75
t[3]	242070DB
t[4]	C1BDCEEE
t[5]	F57C0FAF
…	
t[64]	EB86D391

Алгоритъм MD5 за хеширане на входно съобщение P

- MD5 осигурява необходимата защита от разкриване на явния текст и изпълнява изискванията за избягване на колизии.
- Задачата за намиране на две съобщения с еднакъв хеш-код е с изчислителна сложност от порядъка на 2^{64}, а задачата за намиране на явния текст по зададен хеш-код е 2^{128}, което е практически неизпълнимо за разумно време.

Хеш-алгоритъм SHA-1

- Входното съобщение – **до 2^{64} бита;**
- Изходният хеш-код е **160 бита** (при MD5 – изходният хеш-код е **128** бита);
- Съобщението P се разделя на блокове с размер **512** бита;
- Инициализират се **пет 32-битови променливи**, докато при MD5 са четири;
- SHA-1 се опредеря като **сигурен**, защото е невъзможно възстановяване на съобщението P от неговия хеш-код h(P). Затова SHA-1 се използва в стандарта за електронен подпис DSS, което е и неговото основно предназначение.
- Съобщението трябва да е кратно на 512, при къс блок се извършва допълването му.

Хеш-алгоритъм SHA-1

- Инициализират се следните променливи:

A=67452301;

B=efcdab89;

C=98badcfe;

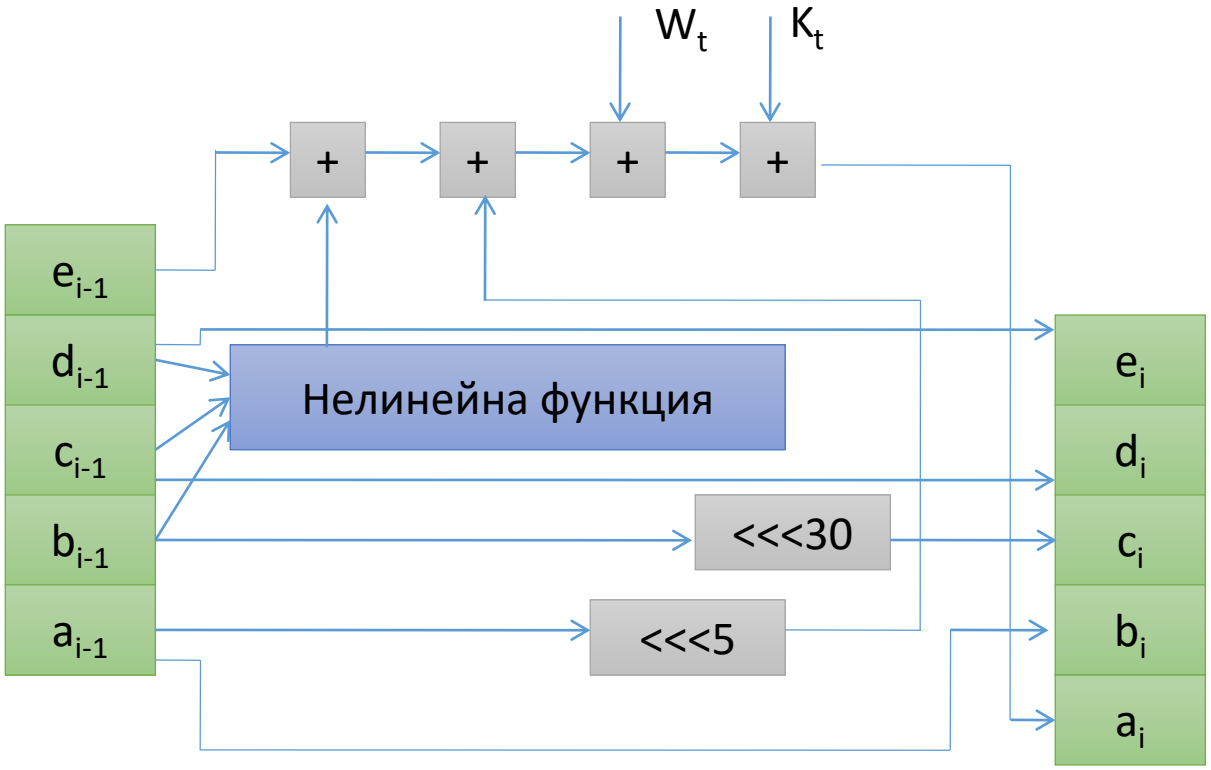
D=10325476;

E=c3d2e1f0.

- Променливите A,B,C,D, E се копират в помощните променливи a,b,c,d,e.
- Главният цикъл обработва последователно блоковете на съобщението: P=P(1),P(2),…P(n).
- Цикълът се състои от **четири фази**, във всяка се изпълняват по **20 операции**.
- Всяка от тези операции използва **нелинейна функция**, с която обработва три от петте променливи.
- Извършва се циклично ляво преместване и сумиране.

Хеш-алгоритъм SHA-1

- Схема на операция в SHA-1:



Хеш-алгоритъм SHA-1

Нелинейните функции в алгоритъма имат вида (t е номерът на операцията):

$$f_t(b,c,d) = (b \wedge c) \vee (b \wedge d), 0 \le t \le 19;$$

$$f_t(b,c,d) = b \oplus c \oplus d, 20 \le t \le 39;$$

$$f_t(b,c,d) = (b \wedge c) \vee (b \wedge d) \vee (c \wedge d), 40 \le t \le 59;$$

$$f_t(b,c,d) = b \oplus c \oplus d, 60 \le t \le 79;$$

Използват се следните константи:

K_t=5a827999, за 0<=t<=19;

K_t=6ed9eba1, за 20<=t<=39;

K_t=8f1bbcdc, за 40<=t<=59;

K_t=ca62c1d6, за 60<=t<=79.

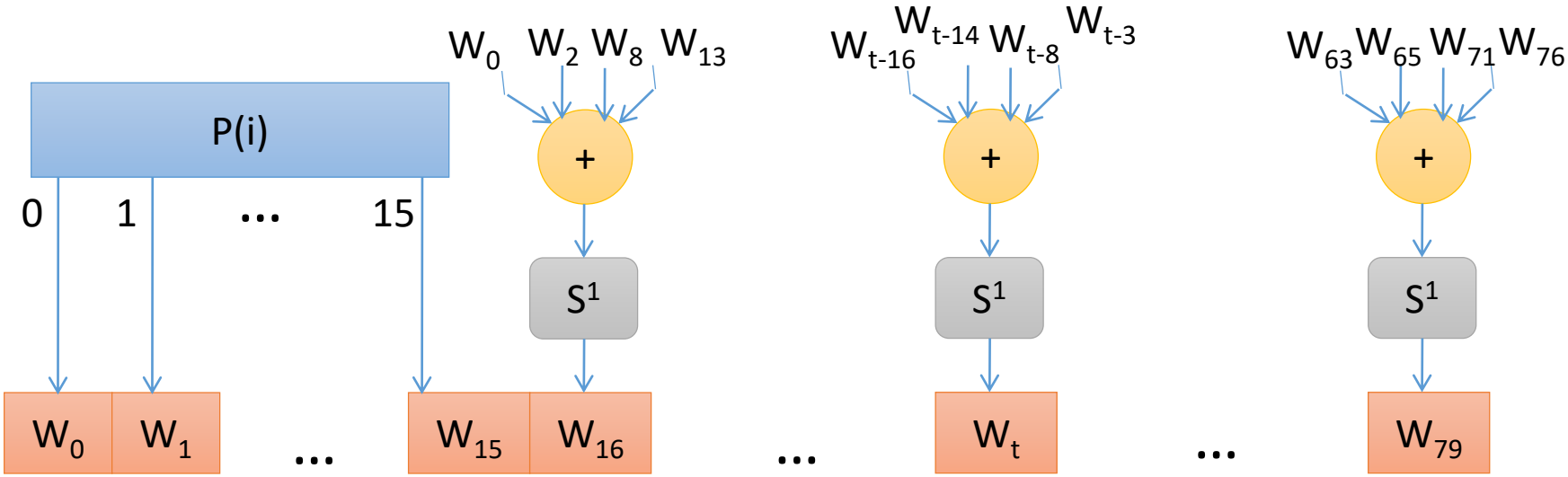
Хеш-алгоритъм SHA-1

- Всеки блок от съобщението P(i), за i=0-15 се преобразува от 16 на брой 32-битови думи в 80 на брой 32-битови думи (W_0-W_{79}).
- Първите 16 подблока от разширеното съобщение се вземат от текущия блок, а останалите се определят по формулата:

$$W_t = (W_{t-16} \oplus W_{t-14} \oplus W_{t-8} \oplus W_{t-3})S^1$$

където W_t е подблок от разширеното съобщение.

Схема за трансформация на входния блок в разширен блок



Сравнение на SHA функциите

Алгоритъм и вариант		Размер на изхода (битове)	Начално състояние размер (битове)	Размер на блоковете (битове)	Максимално съобщение дължина (битове)	Дължина на думата (битове)	Итерации	Използвани операции	Намерени Колизии?
SHA-0		160	160	512	$2^{64} - 1$	32	80	+, &, |, ^, ротация	Да
SHA-1									Теоретична атака (2^{60})
SHA-2	*SHA-224* *SHA-256*	224 256	256	512	$2^{64} - 1$	32	64	+, &, |, ^, >>, ротация	Няма
	SHA-384 *SHA-512* *SHA-512/224* *SHA-512/256*	384 512 224 256	512	1024	$2^{128} - 1$	64	80	+, &, |, ^, >>, ротация	Няма
SHA-3		224/256/384/512	1600 (5×5 масив от 64-битови думи)			64	24	&,^, ~, ротация	Няма

Електронен подпис

- С използване на електронни документи се налага подписването им, с което се гарантира произходът на информацията и връзката им с неговия подател.
- Понастоящем за генериране на ЕП се използват асиметрични криптографски алгоритми – с най-голямо значение са алгоритмите **RSA** (Rivest, Shamir, Adleman) и **DSA** (Digital Signature Algorithm).

Електронен подпис

В България **Законът за електронния документ и електронните удостоверителни услуги** е в сила от 6 октомври 2001г. Предвидени са три вида ЕП:

- **Обикновен ЕП** – предназначен е за комуникация между двойка потребители, които постигат предварително съгласие относно криптографския алгоритъм, използва се в рамките на дадена организация или в дребния бизнес;
- **Усъвършенстван ЕП** – осигурява се по-надеждна защита във финансовата и търговската сфера;
- **Универсален ЕП** – с най-висока сигурност, използва се от държавните органи и институции;

- Усъвършенстван или универсален ЕП може да се използва след получаване на съответни удостоверения от **Доставчик на удостоверителни услуги**.

Доставчици на удостоверителни услуги в България

„Борика“ АД,
„Евротръст Технолъджис“ АД,
„Инфонотари“ ЕАД и
„Информационно обслужване“ АД

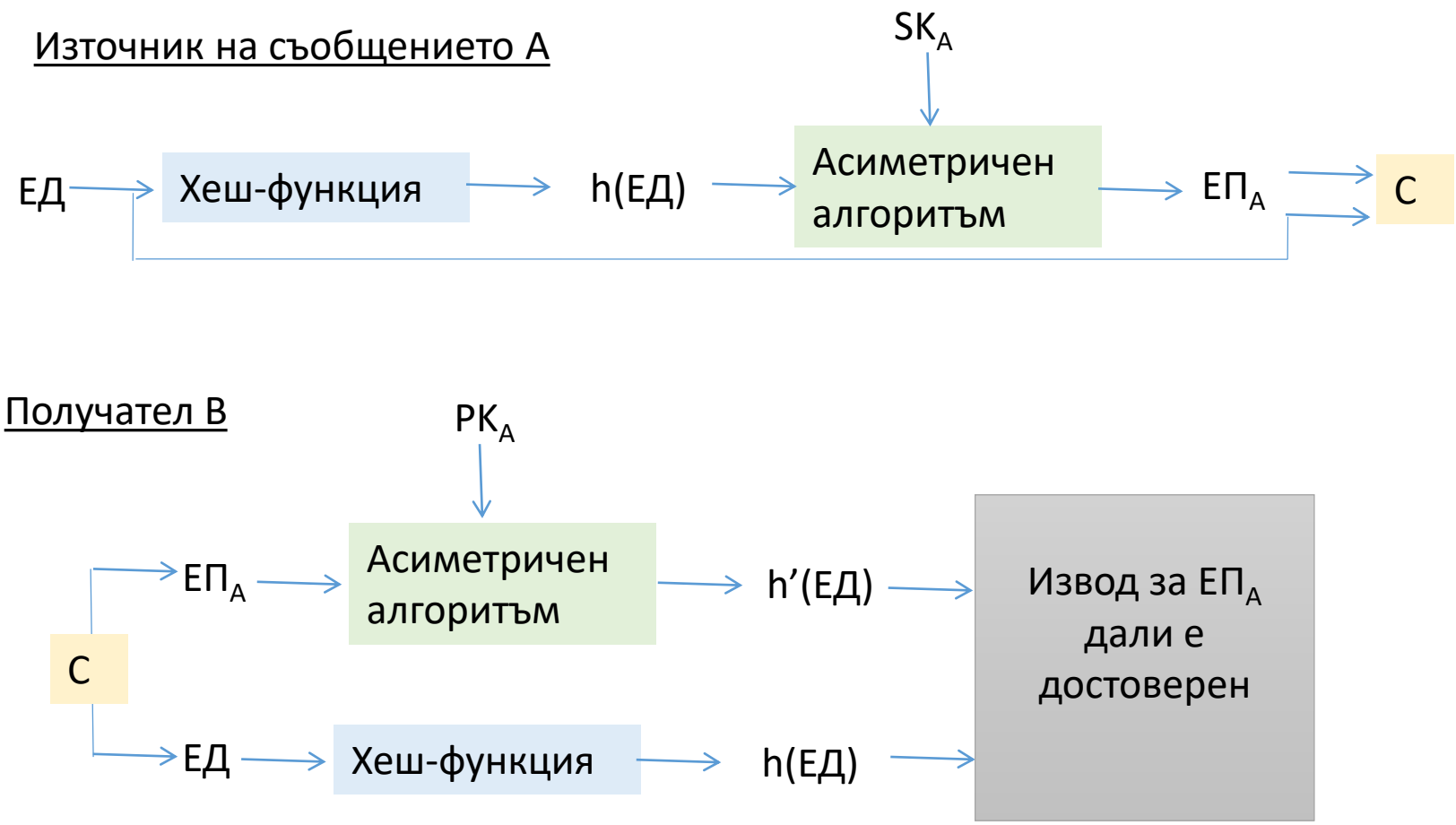
Системи за електронен подпис

- Две групи:
  - **Системи за ЕП с приложение** (СЕПП) – използват оригиналното съобщение като вход на алгоритъма за проверка и хеш-функция за обработка на електронния документ – DSA.
  - **Системи за ЕП с възстановяване на съобщението** (СЕПВС) – подписаното съобщение може да бъде възстановено от ЕП и за проверка на ЕП не се изисква наличието на оригиналния текст – RSA.

Защо се използват хеш-функции в криптографските системи?

- Извършва се **обработка на входен текст с голям обем до изходен хеш-код** – например при SHA-1 входното съобщение е 2^{64} бита, а хеш-кода е 160 бита.
- **Отстранява се излишъкът от входния текст**, което води до **по-висока устойчивост** на криптираното съобщение спрямо атаки.
- **Съкращава се времето** за обработка и се **намалява мрежовият трафик**.

Генериране на ЕП с асиметричен алгоритъм и хеш-функция



Цифрови сертификати

Цифрови сертификати

- В криптографията **X.509** е стандарт на Международния съюз по телекомуникации, определящ формата на сертификатите за публичен ключ – за удостоверяване на човек или устройство и криптиране на комуникация.
- Сертификатите **X.509** се използват в много интернет протоколи, включително TLS/SSL, който е основата за HTTPS.
- Те се използват и като **електронни подписи**.

Цифрови сертификати

- Сертификат X.509 **обвързва идентичност с публичен ключ с помощта на цифров подпис**.
- Сертификатът съдържа самоличност (име на хост, организация или физическо лице) и публичен ключ (RSA, DSA, ECDSA, ed25519 и т.н.) и е издаден/подписан от **сертифициращ орган** или е самоподписан.
- Когато даден сертификат е подписан от доверен сертифициращ орган или е валидиран по друг начин, този, който притежава сертификата, може да
  - **използва публичния ключ, който съдържа, за да установи защитени комуникации** с друга страна или
  - **да валидира документи, цифрово подписани** чрез съответния частен ключ.

Цифрови сертификати

- **Сертификати с публичен ключ**
- **Сертификатът съдържа четири основни части с** информация:
  - **Име на собственика**. Името на собственика на публичния ключ. Този собственик може да бъде човек, устройство, или дори роля в организация.
  - Форматът на това име ще зависи от приложението, но трябва да бъде уникална самоличност, която идентифицира собственика в средата, в която ще бъде използван публичен ключ.

Цифрови сертификати

- **Стойност на публичния ключ**. Самият публичен ключ. Това често е придружено от идентификатор на криптографски алгоритъм, с който публичният ключ е предназначен за използване.
- **Период на валидност**. Това идентифицира датата и часа, от които публичният ключ е валиден и каква е датата и часът на изтичането му.
- **Подпис.** Създателят на сертификата за публичен ключ цифрово подписва всички данни, които формират сертификата за публичен ключ, включително името на собственика, стойността на публичния ключ и периода на валидност.

- Тогава, **цифровият подпис** не само свързва всички тези данни заедно, но е и гаранцията, че създателят на сертификат смята, че всички данни са верни.

Сертификат X.509 v.3

- Повечето сертификати с публичен ключ съдържат много повече информация от тази, като точното съдържание е продиктувано от формата на сертификата, избран за използване в системата за управление на публичен ключ.
- Най-известният формат на сертификата за публичен ключ е **X.509 версия 3.**

Сертификат X.509 v.3

Field	Description
Version	Specifies the X.509 version being used (in this case V3).
Serial Number	Unique identifier for the certificate.
Signature	Digital signature algorithm used to sign the certificate.
Issuer	Name of the creator of the certificate.
Validity	Dates and times between which the certificate is valid.
Subject	Name of the owner of the certificate.
Public-key Info.	Public-key value; Identifier of public-key algorithm.
Issuer ID	Optional identifier for certificate creator.
Subject ID	Optional identifier for certificate owner.
Extensions	A range of optional fields including: *Key identifier* (in case owner owns several public keys); *Key usage* (specifies usage restrictions); *Location of revocation information*; *Identifier of policy relating to certificate*; *Alternative names for owner*.

```
Certificate:
   Data:
       Version: 3 (0x2)
       Serial Number:
           10:e6:fc:62:b7:41:8a:d5:00:5e:45:b6
       Signature Algorithm: sha256WithRSAEncryption
       Issuer: C=BE, O=GlobalSign nv-sa, CN=GlobalSign Organization Validation CA - SHA256 - G2
       Validity
           Not Before: Nov 21 08:00:00 2016 GMT
           Not After : Nov 22 07:59:59 2017 GMT
       Subject: C=US, ST=California, L=San Francisco, O=Wikimedia Foundation, Inc.,
CN=*.wikipedia.org
       Subject Public Key Info:
           Public Key Algorithm: id-ecPublicKey
               Public-Key: (256 bit)
           pub:
                   00:c9:22:69:31:8a:d6:6c:ea:da:c3:7f:2c:ac:a5:
                   af:c0:02:ea:81:cb:65:b9:fd:0c:6d:46:5b:c9:1e:
                   9d:3b:ef
               ASN1 OID: prime256v1
               NIST CURVE: P-256
       X509v3 extensions:
           X509v3 Key Usage: critical
               Digital Signature, Key Agreement
           Authority Information Access:
               CA Issuers - URI:http://secure.globalsign.com/cacert/gsorganizationvalsha2g2r1.c
rt
               OCSP - URI:http://ocsp2.globalsign.com/gsorganizationvalsha2g2
           X509v3 Certificate Policies:
               Policy: 1.3.6.1.4.1.4146.1.20
```

Цифрови сертификати

- **СЪЗДАТЕЛИ НА СЕРТИФИКАТИ**
- Създателят на сертификат играе изключително важна роля, тъй като този създател, като подписва сертификата, **предоставя гаранция, че всички данни, свързани с публичния ключ (включително името на собственика) са правилни**.
- Създателят на сертификат с публичен ключ се нарича **сертифициращ орган (СО)**.
- Сертифициращят орган изпълнява три важни роли:

Цифрови сертификати

- **Създаване на сертификат**. СО поема отговорност за осигуряване на коректна информацията в сертификата, преди да създаде и подпише сертификата и след това да го издаде на собственик.
- **Анулиране на сертификат**. СО е отговорен за анулиране на сертификата в случай, че той става невалиден (да кажем, че му е изтекъл срокът).
- **Доверителна роля на сертификата**. СО действа като точка на доверие за всяка страна, разчитаща на коректността на информацията, съдържаща се в сертификата. За да изпълни тази роля, СО ще трябва активно да поддържа профила си на доверена организация. Може също да се наложи да влезе в отношения с други организации, за да се улесни по-широкото признаване на това доверие.

Цифрови сертификати

- **ТЕХНИКИ ЗА ОТМЕНЯНЕ**
- **Анулирането на сертификати** може да се извърши само по един от трите начина:
  - **Черен списък**. Това включва поддържане на база данни със серийни номера на **сертификати, които са били отменени**. Този тип база данни често се нарича **списък с анулирани сертификати**. Тези анулирани сертификати трябва да се поддържат внимателно, обикновено от CO, който отговаря за тях, с ясни указания колко често се актуализират.

Цифрови сертификати

- **Бели списъци**. Това включва поддържане на база данни със серийни номера на **сертификати, които са валидни**.
  - След това, тази база данни може да бъде използвана за запитвания от доверяваща се страна, за да разбере дали има сертификат, който е валиден.
  - Това е особено полезно за приложения, изискващи информация за статуса на сертификат в реално време.
- **Бързо отменяне**. Това премахва необходимостта от анулиране чрез разпределяне на много кратък живот на сертификатите. Това, разбира се, идва с цената на изискването сертификатите да бъдат преиздавани редовно.